Dr. Subhash University

Innovative Task / Project

Student Grade Tracker

A web-based application for recording marks, calculating grades, and analysing class performance

Subject: Software Engineering And Testing

(2BTCO501)

Subject Incharge

Prof. Ami Mehta

_____

(Signature)

Team Member

1. [Your Name]

([Enrollment Number])

Academic Year: 2026 – 2027

Submitted to

Dr. Subhash University, Junagadh

Index

Sr. No.	Contents	Page No.
1	Screenshots of the Developed System	3
2	A. Problem Statement	6
	Student Grade Tracking, Objectives and Project Overview	6
3	B. Software Requirements Specification (SRS)	6
	1. Introduction	6
	2. Overall Description	7
	3. Specific Requirements	8
	4. System Architecture Overview	9
	5. Risk Analysis	9
4	C. Model for Project: Spiral Model	9
5	D. Testing Techniques Used	10
6	E. Observations and Conclusion	11
7	Evaluation	12

List of Figures

Figure	Title	Page No.
1	Dashboard with summary cards and charts	3
2	Manage Students and Grades page	3
3	Inline validation for a duplicate roll number	3
4	Delete confirmation modal	4
5	Individual student report card	4
6	Class ranking and data actions	4
7	Mobile responsive dashboard	5
8	System architecture of the Student Grade Tracker	9

List of Tables

Table	Title	Page No.
1	Screenshots Mapped to SRS Requirements	5
2	Functional Requirements	8
3	Non-Functional Requirements	8
4	Interface Requirements	8
5	Risk Analysis and Mitigation	9
6	Application of the Spiral Model	9
7	Testing Techniques Used	10
8	Representative Test Scenarios	11
9	Observations from the Developed System	11
10	Evaluation Checklist	12

Screenshots of the Developed System

The figures below present the working screens of the Student Grade Tracker. Each screen was captured from the final build after validation, persistence, and responsive-layout testing. Table 1 maps every screen to the SRS requirements that it demonstrates.

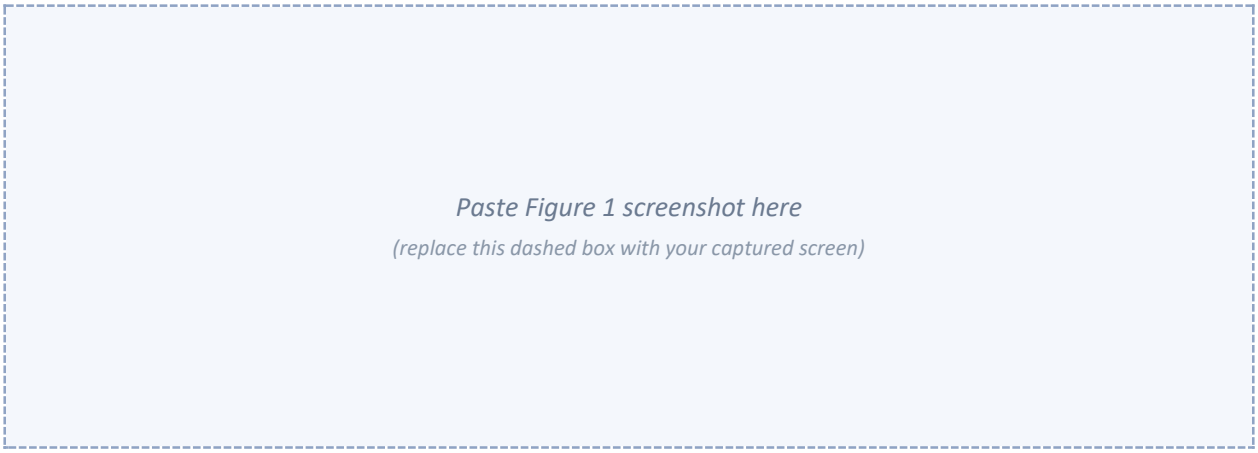


Figure 1: Dashboard with summary cards and charts

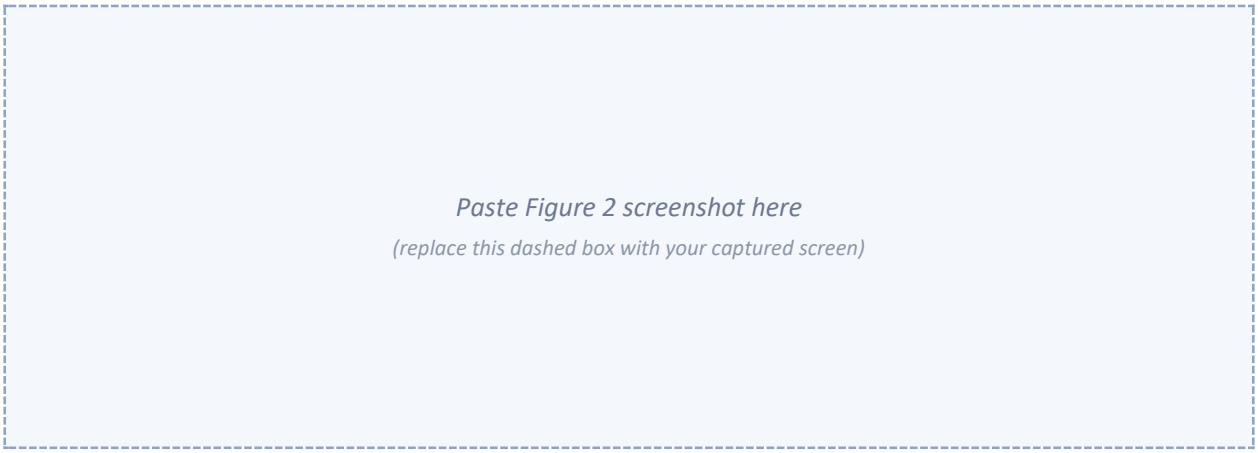


Figure 2: Manage Students and Grades page

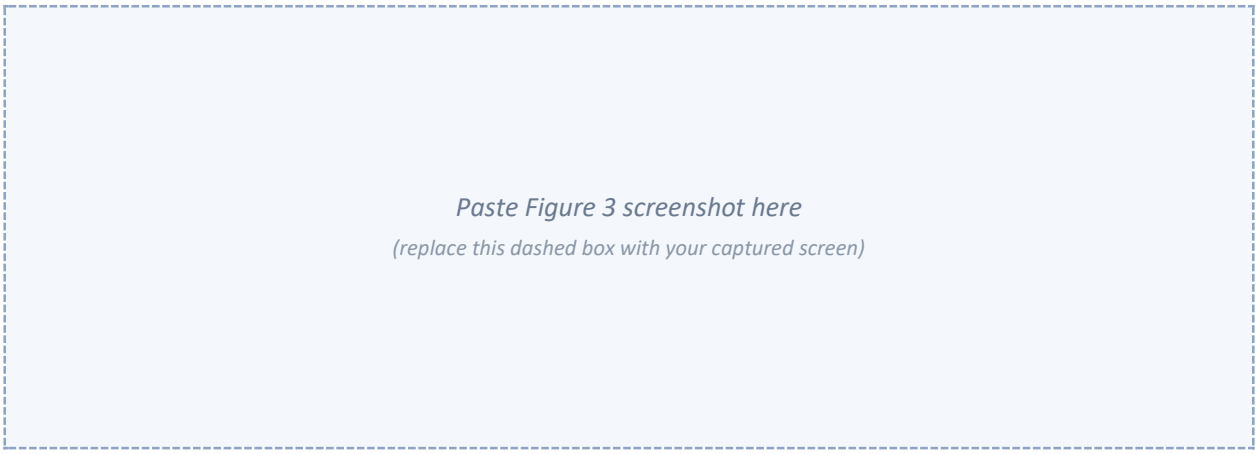


Figure 3: Inline validation for a duplicate roll number

Paste Figure 4 screenshot here

(replace this dashed box with your captured screen)

Figure 4: Delete confirmation modal

Paste Figure 5 screenshot here

(replace this dashed box with your captured screen)

Figure 5: Individual student report card

Paste Figure 6 screenshot here

(replace this dashed box with your captured screen)

Figure 6: Class ranking and data actions

Paste Figure 7 screenshot here

(replace this dashed box with your captured screen)

Figure 7: Mobile responsive dashboard

Table 1: Screenshots Mapped to SRS Requirements

Fig.	Screen	What It Demonstrates	Related Requirement
1	Dashboard	Summary cards, grade distribution doughnut chart, subject averages, top five performers, and the at-risk list.	FR7, NFR1, NFR2
2	Manage Students	Add-student form, five subject inputs, total and percentage, grade and status chips, search, grade filter, and sorting.	FR1, FR3, FR5, FR6
3	Validation	Required-field checks, duplicate roll number detection, and 0–100 range validation with inline error messages.	FR2
4	Delete Confirmation	Modal confirmation before a record is permanently removed from storage.	FR4
5	Report Card	Student details, subject bar chart, marks table, total, percentage, grade, and descriptive remark.	FR8
6	Ranking and Actions	Class ranking with tie handling, CSV export, report printing, and reset-to-sample-data controls.	FR9, FR10
7	Mobile Layout	Responsive single-column layout at a 390 px viewport with no horizontal page overflow.	NFR1, NFR6

A. Problem Statement

Student Grade Tracking

Educational institutions maintain academic records for many students across several subjects. When these records are kept in registers or spreadsheets, calculating totals, percentages, grades, and pass/fail status for every student becomes repetitive and error-prone. A single formula mistake can produce an incorrect grade, while locating a particular result or preparing a class ranking consumes a considerable amount of time. The Student Grade Tracker addresses this problem with a focused web application that stores student records locally in the browser and applies fixed academic rules consistently.

The system must:

- **Allow a teacher to add, edit, and delete student records** with marks for five subjects.
- **Validate all input,** including required fields, numeric ranges, and unique roll numbers.
- **Calculate total, percentage, grade, and pass/fail status** using transparent rules.
- **Present class-level information** through summary cards, charts, and a ranked list.
- **Generate an individual report card** that can be printed or saved as a PDF.
- **Retain data between sessions** using browser local storage.

Objectives

- **Eliminate manual calculation errors** by keeping grade and pass rules in one configuration object.
- **Reduce the time needed** to record marks, find a student, and interpret a result.
- **Provide an immediate class overview** including average performance, grade distribution, and at-risk students.
- **Demonstrate a complete engineering process** from requirements and design to implementation, testing, and evaluation.
- **Keep the application lightweight** and easy to run from any modern browser without a backend.

Project Overview

The application is implemented as three static HTML pages: a dashboard, a manage-students page, and a reports page, supported by shared CSS and JavaScript modules. Records are stored as a JSON array under the localStorage key studentGradeTracker.students. Ten sample records are seeded on first use so that every screen is populated for demonstration. The dashboard visualises class statistics, the manage page performs CRUD operations, and the reports page generates a report card, class ranking, CSV export, print output, and a reset-to-sample-data action.

B. Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This SRS defines the requirements for the Student Grade Tracker, a browser-based application that helps a teacher record marks for one class, calculate results automatically, and analyse class performance. It is intended for the project team, the subject incharge, and evaluators who will verify that the delivered software satisfies its stated scope.

1.2 Scope

The system covers student record management, automatic result calculation, class analytics, individual report cards, ranking, CSV export, and printing. It is a single-user, client-side application; user accounts, server-side storage, attendance, fees, and official university result integration are outside its scope.

1.3 Definitions

- **Student:** A person whose name, roll number, semester, and subject marks are recorded.
- **Roll Number:** A unique identifier assigned to a student within the class.
- **Total:** The sum of the five subject marks, out of 500.
- **Percentage:** Total marks divided by 500, multiplied by 100.
- **Grade:** A letter derived from percentage: A (80+), B (70–79.99), C (60–69.99), D (40–59.99), or F (below 40).
- **Pass:** A status awarded when every subject mark is at least 35 and the overall percentage is at least 40.
- **Report Card:** A printable summary of one student's marks, total, percentage, grade, and remark.
- **Seed Data:** Ten sample students loaded on first use or after a reset.

2. Overall Description

2.1 Product Perspective

The product is a standalone web application with no server component. Student records are kept in the browser's localStorage, while Chart.js is loaded from a CDN for the dashboard and report charts. All calculations are performed in JavaScript. The application can be opened directly from the file system or served from any static web server.

2.2 Product Functions

- Create, read, update, and delete student records.
- Validate names, roll numbers, semesters, and five subject marks.
- Calculate totals, percentages, grades, pass/fail status, and class ranks.
- Search, filter, and sort the class list.
- Visualise grade distribution, subject averages, top performers, and at-risk students.
- Print a report card, export data as CSV, and restore sample data.

2.3 User Classes and Characteristics

The primary user is a teacher or evaluator who records marks and interprets results. Students are secondary users who may view or print their own report card. Developers and testers maintain the code and verify behaviour. The interface assumes basic familiarity with web forms and tables; no technical knowledge of grading rules is required.

2.4 Operating Environment

The application runs in recent versions of Chrome, Edge, Firefox, and Safari on Windows, macOS, Linux, Android, and iOS. It requires a viewport of at least 360 pixels and has no build step or runtime dependency other than the Chart.js CDN. A modern browser with localStorage enabled is required.

2.5 Assumptions and Dependencies

- **One class and one set of five subjects** are managed at a time.
- **All marks are entered out of 100** and are stored as numbers.
- **localStorage is available and belongs to one browser profile;** data is not shared across devices.
- **The Chart.js CDN is reachable** when charts are displayed.
- **Seed data represents a demonstration class** and can be replaced by real records.

3. Specific Requirements

3.1 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Priority	Module
FR1	Add a student with name, unique roll number, semester, and marks for five subjects.	High	Student entry
FR2	Validate required fields, numeric marks from 0 to 100, and unique roll numbers; show inline errors.	High	Data integrity
FR3	Edit an existing record and immediately recalculate its result, status, and rank.	High	CRUD
FR4	Delete a record only after explicit confirmation in a modal dialog.	Medium	Safe deletion
FR5	Calculate total, percentage, grade, and pass/fail using one shared configuration object.	High	Core logic
FR6	Search by name or roll number, filter by grade, and sort by name, percentage, or roll number.	Medium	Usability
FR7	Show dashboard cards, grade distribution, subject averages, top five students, and at-risk students.	High	Analytics
FR8	Display an individual report card with a chart, subject marks, result summary, and remark.	High	Reporting
FR9	Produce a class ranking with correct ties; export CSV and print a report card.	Medium	Reporting
FR10	Persist data in localStorage, recover from corrupted data, and allow reset to sample data.	High	Persistence

3.2 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Requirement	Priority
NFR1	Responsive layout that adapts cleanly to mobile, tablet, and desktop widths.	High
NFR2	Usable and accessible controls: labels, ARIA attributes, focus states, empty states, and toasts.	High
NFR3	Fast interactions: updates and recalculations complete in well under one second for a class-sized dataset.	Medium
NFR4	Reliable persistence with safe recovery from missing or corrupted storage.	High
NFR5	Secure rendering: all user-supplied text is escaped before being inserted into HTML.	High
NFR6	Maintainable structure with separated files, shared utilities, and a single source of truth for rules.	Medium
NFR7	Portable: no build step; runs from the file system in a modern browser.	High

3.3 Interface Requirements

Table 4: Interface Requirements

Interface	Description
User interface	Three responsive pages with a shared navigation bar, page header, cards, tables, and footer.
Input interface	Add/edit form with text, number, and select fields; inline validation messages beneath each field.
Storage interface	localStorage key studentGradeTracker.students holding a JSON array of student objects.
Chart interface	Chart.js doughnut and bar charts rendered inside fixed-height responsive containers.
Export/print interface	CSV download using a Blob; report-card printing isolated through print-specific CSS.

4. System Architecture Overview

The application follows a simple three-layer architecture. The presentation layer contains the three HTML pages and the responsive stylesheet. The business-logic layer contains page controllers and shared utilities that validate input, calculate results, rank students, and escape output. The data layer wraps localStorage access in a small storage module. Chart.js is an external library used only for visualisation.

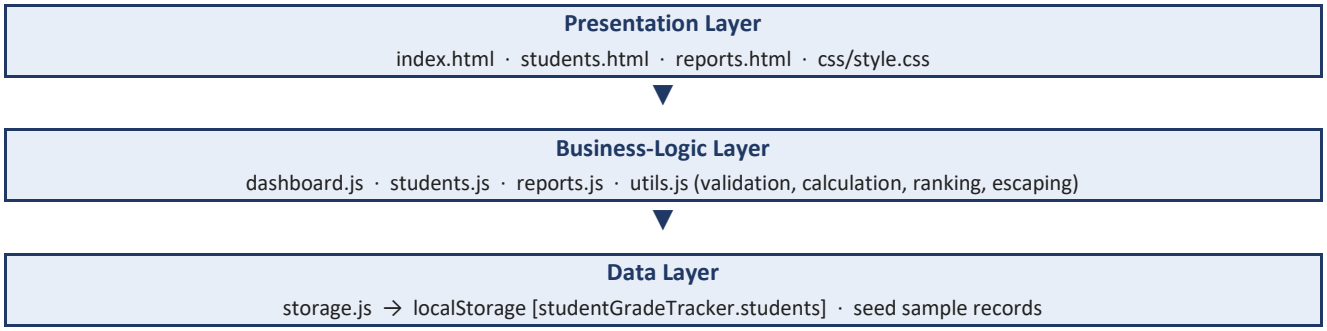


Figure 8: System Architecture of the Student Grade Tracker

The modules communicate through explicit function calls rather than global state. Students are loaded once per page from storage, transformed by shared calculation helpers, and rendered with escaped text. Chart instances are destroyed before recreation to prevent canvas growth. Keeping these responsibilities separate made validation, ranking, and responsive fixes easy to test in isolation.

5. Risk Analysis

The main risks relate to browser-only storage, untrusted input, boundary conditions in academic rules, and layout issues on small screens. Each risk was reviewed during the spiral cycles and addressed with a concrete mitigation.

Table 5: Risk Analysis and Mitigation

ID	Risk	Impact	Mitigation
R1	Data loss when browser storage is cleared.	High	CSV export, documented storage limits, and a reset-to-sample-data action.
R2	Stored XSS through a crafted name or roll number.	High	Escape every user value before HTML insertion; no inline event handlers.
R3	Corrupted or partially written localStorage data.	Medium	try/catch parsing, array validation, numeric fallbacks, and reseeding.
R4	Duplicate roll numbers create ambiguous records.	Medium	Case-insensitive uniqueness check with an inline error.
R5	Incorrect grade or pass/fail at a boundary.	High	One GRADE_CONFIG source of truth and explicit boundary tests.
R6	Charts grow or overflow their cards.	Medium	Fixed-height chart containers; destroy and recreate chart instances.
R7	Layout breaks on small screens.	Medium	Responsive breakpoints verified at 390, 768, and 1440 pixels.

C. Model for Project: Spiral Model

The Spiral model was selected because the project evolved through repeated cycles of planning, risk analysis, implementation, and evaluation. Each cycle produced a working increment, which was reviewed before the next cycle began. This suited a small application whose requirements grew from basic record keeping to dashboards, reports, exports, and responsive behaviour.

Table 6: Application of the Spiral Model

Spiral Phase	Activities Performed in This Project	Outcome
1. Determine objectives	Identify the users, core grading rules, CRUD scope, and the three-page structure.	Agreed scope and success criteria.
2. Identify and resolve risks	Review storage loss, duplicate rolls, grade boundaries, and XSS; choose mitigations.	Risk register and validation strategy.
3. Develop and test	Build the data module, calculation helpers, student form, table, and core tests.	Working CRUD build with verified calculations.
4. Plan the next iteration	Add the dashboard, charts, ranking, report card, and export/print actions.	Second working increment.
5. Evaluate the result	Test boundaries, persistence, responsive layouts, and chart behaviour; fix defects.	Stable build and revised requirements.
6. Review with the stakeholder	Check the screens against the SRS and the report requirements; refine wording and layout.	Final build and project report.

The iterative approach was useful for three reasons. First, risky areas such as grade boundaries and storage recovery were tested early. Second, each new feature could be added without rewriting the calculation core. Third, feedback on the interface and the charts led to focused fixes rather than a large redesign. The completed cycles delivered CRUD and grading, analytics and ranking, reporting and export, and finally reliability, security, and responsive polishing.

D. Testing Techniques Used

Testing combined white-box checks of the shared calculation helpers with black-box testing of the user interface. The goal was to verify both the correctness of academic rules and the behaviour of the application under invalid, boundary, and recovery scenarios.

Table 7: Testing Techniques Used

Technique	Purpose	Where Applied
Unit testing	Verify individual calculation functions and configuration values.	Grade and pass logic, ranking helper.
Black-box functional testing	Exercise features through the UI without inspecting internals.	Add, edit, delete, search, filter, sort, export.
White-box testing	Trace conditional paths and shared references in the code.	Pass rule, grade thresholds, storage fallbacks.
Boundary value analysis	Test values at and around rule thresholds.	35, 40, 60, 70, 80; marks 0 and 100.
Equivalence partitioning	Group inputs into valid and invalid classes.	Unique vs duplicate rolls; numeric vs blank marks.
Negative testing	Confirm graceful rejection of invalid input.	Blank fields, out-of-range marks, duplicate roll.
Integration testing	Check consistency across storage, pages, charts, and reports.	Form → table → dashboard → report card.
Regression testing	Re-run critical checks after every change.	Calculations after chart and responsive fixes.

Technique	Purpose	Where Applied
Responsive/UI testing	Inspect layout and overflow at representative widths.	390, 768, and 1440 pixel viewports.
Persistence and security testing	Verify recovery and safe rendering.	Reload, corrupted JSON, reset, XSS payload.

Representative Test Scenarios

Table 8: Representative Test Scenarios

ID	Scenario / Input	Expected Result	Req.
TS1	Add a valid student with all five subject marks.	Record is saved; total, percentage, grade, and status are correct; success toast appears.	FR1, FR5
TS2	Add a student using an existing roll number (including a different letter case).	Inline duplicate-roll error appears; record is not saved.	FR2
TS3	Submit blank name, blank marks, or marks above 100.	Submission is blocked and each invalid field shows a message.	FR2
TS4	Enter marks of 0 and 100.	Values are accepted; percentage is 0 or 100; grade is F or A.	FR2, FR5
TS5	Enter percentages at 79.99, 80, 69.99, 70, 59.99, 60, 39.99, and 40.	The correct grade is assigned at every boundary.	FR5
TS6	One subject is 34.99 while the overall percentage is at least 40.	Overall status is Fail; all subjects at 35 or above with 40+ overall passes.	FR5
TS7	Edit a student's marks and save.	Table, summary cards, charts, and rank update consistently.	FR3
TS8	Click Delete, then Cancel; repeat and confirm.	Cancel keeps the record; confirmation removes it and persists the change.	FR4
TS9	Search by partial name/roll, filter by grade, and sort each column.	The correct subset and order are shown.	FR6
TS10	Create two students with equal percentages.	Both share the same rank and the next rank skips accordingly (1, 1, 3).	FR9
TS11	Reload, corrupt localStorage, then reset sample data.	Data persists across reload; corrupted data is replaced safely; reset restores ten samples.	FR10
TS12	Export CSV and print a report card.	CSV contains all records; print view shows only the selected report card.	FR9

All twelve scenarios were executed manually against the final build. The grade and pass boundaries, duplicate-roll handling, storage recovery, escaping of HTML in names, and the mobile layout were also checked with scripted smoke tests and browser developer tools. No open defects remain in the delivered scope.

E. Observations and Conclusion

The final build was evaluated with the ten seeded students. The observations below summarise the class-level results produced by the application and the behaviour verified during testing.

Table 9: Observations from the Developed System

Parameter	Observation
Students in sample data	10
Aggregate marks	3344 / 5000
Class average	66.88%
Highest percentage	91.80% (Aarav Sharma)
Pass percentage	80% (8 of 10 students)
Grade distribution	A = 3, B = 2, C = 1, D = 2, F = 2
Subject averages	Mathematics 66.5, Physics 67.2, Chemistry 66.1, Biology 66.4, Computer Science 68.2
At-risk students	2 students below 40% overall, listed automatically on the dashboard
Ranking behaviour	Equal percentages share a rank; the next rank skips (verified: 1, 1, 3)
Responsive behaviour	No horizontal overflow at 390 px; charts remain inside their cards
Data recovery	Reload, corrupted JSON, empty storage, and reset all behaved as specified

The results confirm that the application performs its core task reliably: marks are validated, totals and percentages are accurate, grades follow the configured thresholds, and pass/fail status respects both the per-subject minimum and the overall minimum. The dashboard and report card make the same results easy to interpret, and the ranking demonstrates correct tie handling. Security and persistence checks also passed: HTML in a student name is escaped, and the storage module recovers gracefully from missing or malformed data.

Conclusion

The Student Grade Tracker meets its stated objectives. It replaces manual tabulation with a small, transparent application that records students, calculates results consistently, presents class analytics, and produces printable report cards. Building it demonstrated the value of a single source of truth for business rules, validation at the boundary of the system, defensive storage access, and repeated testing at grade and pass boundaries.

The main limitations are the lack of a server database and multi-user access, the fixed set of five subjects, and dependence on browser storage. Future work could add a backend with authentication, support for multiple classes and semesters, configurable subjects, import from spreadsheets, and richer analytics. Within its intended single-user scope, however, the application is complete, tested, and ready for demonstration.

Evaluation

The project was reviewed against the requirements and the quality criteria below. Each item was verified in the final build and through the test scenarios in Table 8.

Table 10: Evaluation Checklist

Criteria	Status	Remarks
Requirements coverage	Complete	FR1–FR10 implemented and traced to screens.
User interface and responsiveness	Complete	Consistent theme; verified at 390, 768, and 1440 px.
Validation and error handling	Complete	Required, range, duplicate, corruption, and empty states.
Security	Complete	All user text escaped; no inline handlers or exposed secrets.
Data persistence and recovery	Complete	localStorage, safe fallback, and reset-to-sample action.
Reports and exports	Complete	Report card, ranking, CSV export, and print layout.
Code structure and maintainability	Complete	Separated modules, shared config, and reusable helpers.
Testing and edge cases	Complete	Twelve scenarios plus boundary and responsive checks.
Documentation	Complete	README, assumptions, data model, and this report.
Overall readiness	Ready	Suitable for demonstration and viva.

Remarks: _____

_____

Subject Incharge
Prof. Ami Mehta

Signature
_____